

Data Governance & Unity Catalog

1. Overview of Data Governance and Unity Catalog

Data Governance is the set of rules, policies, and processes that control how data is managed, accessed, and used across an organisation. The goal is to make sure data is accurate, secure, and used responsibly.

- Controls who can see and use what data.
- Tracks where data comes from and how it moves (data lineage).
- Ensures compliance with regulations like GDPR.
- Maintains a central record of what data exists and what it means (data catalog).

Unity Catalog is Databricks' built-in data governance solution. It gives you a single place to manage access, audit, and discover all your data assets — across all your Databricks workspaces.

- Centralised access control for tables, files, and ML models.
- Automatic data lineage — see exactly where data came from and where it went.
- Works across multiple workspaces from one place.
- Built-in auditing — logs every access and action.

2. Create Unity Catalog Metastore and Enable a Databricks Workspace with Unity Catalog

What is a Metastore?

A Metastore is the top-level container in Unity Catalog. It stores all metadata — information about your databases, tables, schemas, and permissions. Think of it as the master registry for your data.

How to create a Unity Catalog Metastore:

- Go to the Databricks Account Console (not the workspace).
- Navigate to Data > Create Metastore.
- Give it a name, select a region, and link it to an Azure Data Lake Storage (ADLS) Gen2 container — this is where the actual data will be stored.
- Assign the metastore to one or more workspaces.

Enabling a Workspace with Unity Catalog:

- Once a metastore is created, attach it to your Databricks workspace from the Account Console.
- The workspace must be on the Databricks Premium plan or above.
- After attaching, users in that workspace can access the catalog, create schemas, and manage permissions under Unity Catalog.

3. Overview of 3-Level Namespace and Creating Unity Catalog Objects

Unity Catalog uses a 3-level namespace to organise data. Every object lives inside this structure:

Catalog → Schema (Database) → Table / View / Function

Level 1 — Catalog

The top level. A catalog groups related databases together. You can have multiple catalogs — for example, one for production, one for development.

- **Create:** `CREATE CATALOG my_catalog;`

Level 2 — Schema (Database)

The middle level. A schema sits inside a catalog and holds tables, views, and functions. It's the same as a database in traditional SQL.

- **Create:** `CREATE SCHEMA my_catalog.my_schema;`

Level 3 — Table / View / Function

The bottom level. This is your actual data — stored as managed or external tables, or as views and functions.

- **Managed Table:** `CREATE TABLE my_catalog.my_schema.my_table (id INT, name STRING);`
- **External Table:** Points to data stored at a specific path in ADLS — you manage the storage yourself.

Querying using the 3-level namespace:

```
SELECT * FROM my_catalog.my_schema.my_table;
```

This format makes it clear exactly where the data lives — no ambiguity across workspaces or environments.

Permissions — quick example:

- `GRANT SELECT ON TABLE my_catalog.my_schema.my_table TO user@company.com;`
- You can grant access at any level — catalog, schema, or table — and it flows down.